

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КРЕДИТ НА ЖИЛЬЁ НА ЯЗЫКЕ (ВАРИАНТ 19)

Выполнил:
студент группы 450503
Кузьменков Е.В.

Проверила:
Внук О.М.

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить методы работы с интегрированной средой разработки Microsoft Visual Code на объектно-ориентированном языке Go. Научиться разрабатывать в выбранной среде приложение с графическим интерфейсом на базе фреймворка Fyne. Изучить архитектурный паттерн MVC (Model–View–Controller) в вариантах с активной и пассивной моделью и применить его на практике.

В соответствии с вариантом 19 «Кредит на жильё» решались следующие задачи:

1. Установить и настроить Microsoft Visual Code и комплект разработчика Go, изучить функциональные возможности среды и средства отладки, продемонстрировать работу отладчика.
2. Создать пустой проект под вариант задания, установить расширения для работы с UML и SysML v2 и подобрать аналоги расширений, приведённых в методических указаниях.
3. Разработать приложение с графическим интерфейсом: главная форма с кнопкой «Ввести данные», дополнительное окно ввода (процентной ставки, суммы и срока кредита), отображение результата на главной форме.
4. Обеспечить запоминание последних введённых данных и их восстановление при повторном вводе, а также вывод сообщения об ошибке при некорректных данных.
5. Разработать и описать каркас MVC с активной моделью, построить диаграмму классов.
6. Ответить на контрольные вопросы и оформить отчёт согласно СТП БГУИР.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки (IDE, Integrated Development Environment) представляет собой единую рабочую среду программиста, включающую в себя редактор кода с подсветкой синтаксиса, транслятор для преобразования текста в понятный компьютеру язык, отладчик для поиска недочетов и механизмы автоматической сборки проекта в рабочую программу.

Microsoft Visual Studio Code – IDE от компании Microsoft для языка Go. Основные компоненты среды: Solution Explorer (дерево файлов и проектов), редактор кода, окно Error List со списком ошибок и предупреждений, окно Output для вывода, а также визуальный конструктор (Designer) для графических приложений (Windows Forms, WPF).

2.2 Основные приёмы работы с редактором кода

Редактор Visual Studio Code выполняет фоновый анализ кода: ошибки компиляции подчеркиваются, а комбинация Alt+Enter или Ctrl+. (Quick Actions) предлагает варианты исправления. Автодополнение IntelliSense (Ctrl+Space) подставляет имена классов и методов. Форматирование кода выполняется командой Ctrl+K, Ctrl+D. Навигация: F12 – переход к объявлению, Shift+F12 – поиск использований. 2.3 Сборка, запуск и отладка проекта.

2.3 Сборка, запуск и отладка проекта

Сборка проекта (Build → Build Solution, F6) компилирует изменённые классы. Запуск с отладкой выполняется командой Start Debugging (F5), без отладки – Start Without Debugging (Ctrl+F5). Точка останова (breakpoint) ставится щелчком по полю слева от номера строки (или клавишей F9). При остановке доступны окна Locals и Watch (значения локальных переменных и наблюдаемых выражений). Пошаговое выполнение: Step Over (F10) – выполнить строку, Step Into (F11) – войти в вызываемый метод, Step Out (Shift+F11) – выйти из метода.

2.4 Паттерн MVC

Паттерн MVC (Model-View-Controller) разделяет моделирование предметной области, визуальное отображение и действия пользователя на три компонента. Модель (Model) управляет поведением и данными приложения, реагируя на инструкции по изменению состояния. Представление (View) управляет выводом информации. Контроллер (Controller) интерпретирует ввод пользователя, сообщая модели или представлению о необходимости изменений.

2.5 Активная и пассивная модель

Паттерн MVC предполагает два способа обновления данных. В пассивной модели контроллер эксклюзивно управляет моделью: он вносит изменения, а затем напрямую сообщает представлению о необходимости обновления. Активная модель опирается на паттерн «Наблюдатель» (Observer): модель самостоятельно рассылает уведомления при смене своего состояния. Представление подписывается на эти оповещения и само запрашивает актуальные данные, тогда как контроллер лишь обрабатывает действия пользователя. В данной лабораторной работе активная модель реализована посредством события `StateChanged`.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Вариант 19 требует разработать программу: «Кредит на жильё». Пользователь вводит сумму кредита, годовую процентную ставку и срок. Программа показывает, во сколько раз общая сумма выплат превысит полученную сумму, и определяет ежемесячный платёж. Ввод выполняется в отдельном окне по кнопке «Ввести данные», последние корректные значения восстанавливаются при повторном открытии, а ошибки сопровождаются сообщением.

Среда разработки – Visual Studio Code, язык – Go, графическая библиотека – Fyne v2.7.0.

3.2 Настройка среды и расширений для UML и SysML v2

Установлена среда Microsoft Visual Studio Code. Создан go модуль с названием housing-loan. В файле указана целевая версия go 1.26.4.

Для работы с SysML v2 и PlantUML можно использовать расширения для Visual Studio Code (например, PlantUML Integration) для генерации диаграмм. Структурная модель системы в текстовой нотации SysML v2 приведена на рисунке 3.1.

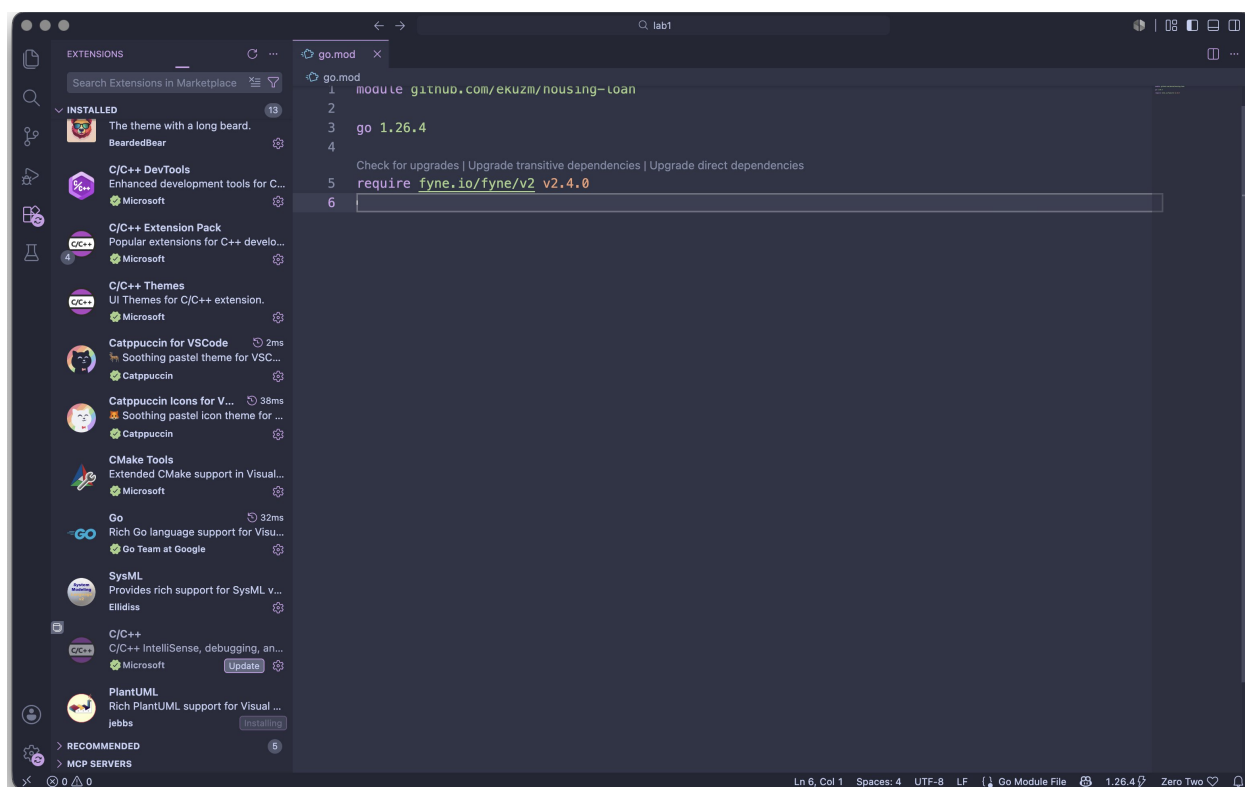


Рисунок 3.1 – Расширения Go, PlantUML и SysML v2 в Visual Studio Code

Структурная модель приложения в нотации SysML v2:

```
package HousingLoan {
  part def LoanModel {
    attribute input : LoanInput;
    attribute result : LoanResult;
    port notify;
  }
  part def MainView {
    port listen;
    port userAction;
  }
  part def FyneInputDialog;
  part def LoanController { port handle; }
  part system {
    part model : LoanModel;
    part view : MainView;
    part dialog : FyneInputDialog;
    part controller : LoanController;
    connect view.userAction to controller.handle;
    connect model.notify to view.listen;
  }
}
```

3.3 Демонстрация отладчика

Работа отладчика продемонстрирована на методах валидации и расчёта ипотечного кредита (validate, calculate). Точка останова установлена в методе Calculate перед вызовом функции валидации. При остановке в окне Locals видны переданные параметры кредита: сумма, годовая процентная ставка и срок в месяцах. Пошаговое выполнение позволяет проконтролировать проверку допустимости введённых данных, вычисление месячной процентной ставки, аннуитетного платежа, общей суммы выплат и переплаты.

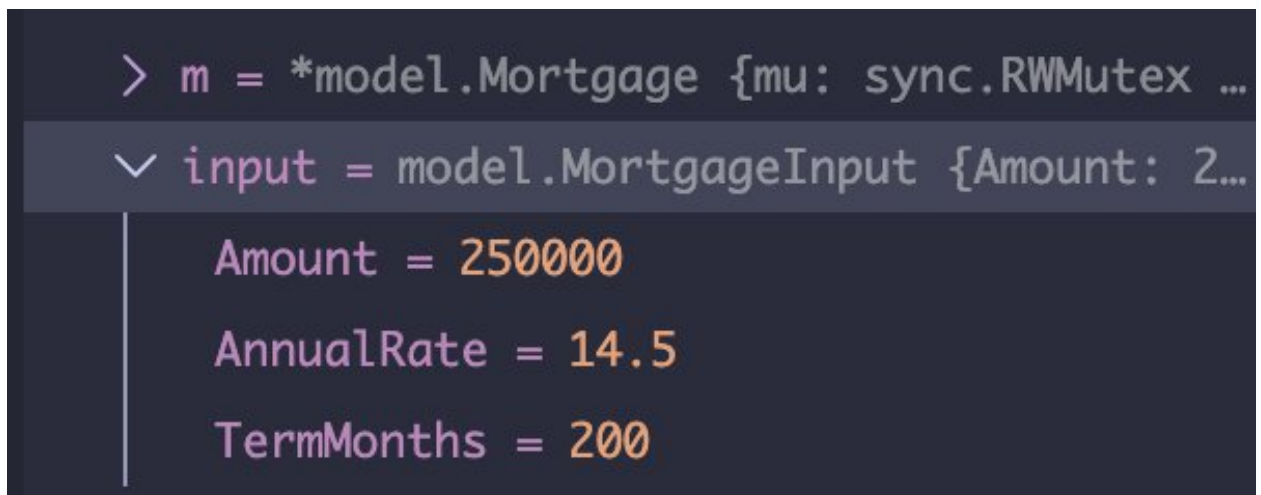


Рисунок 3.2 – Остановка на точке останова, панель Locals

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на пакеты, соответствующие компонентам MVC:

- model: активная модель Mortgage, структуры MortgageInput для введённых данных, MortgageResult для результатов расчёта и State для хранения состояния приложения;
- view: представление Mortgage, содержащее главное окно и окно ввода параметров кредита, а также пользовательскую тему оформления;
- controller: контроллер Mortgage, который преобразует введённые строки в числовые значения и связывает представление с моделью;
- apperror: типизированные ошибки ввода и вычислений;
- app: создание и связывание компонентов приложения.

В main.go вызывается app.Run(). В нём создаются приложение Fyne, модель, контроллер и представление. Представление передаёт введённые данные контроллеру, контроллер проверяет их формат и вызывает модель. Модель выполняет расчёт кредита, сохраняет последнее состояние и уведомляет представление через метод ModelChanged, после чего на главной форме обновляются результаты.

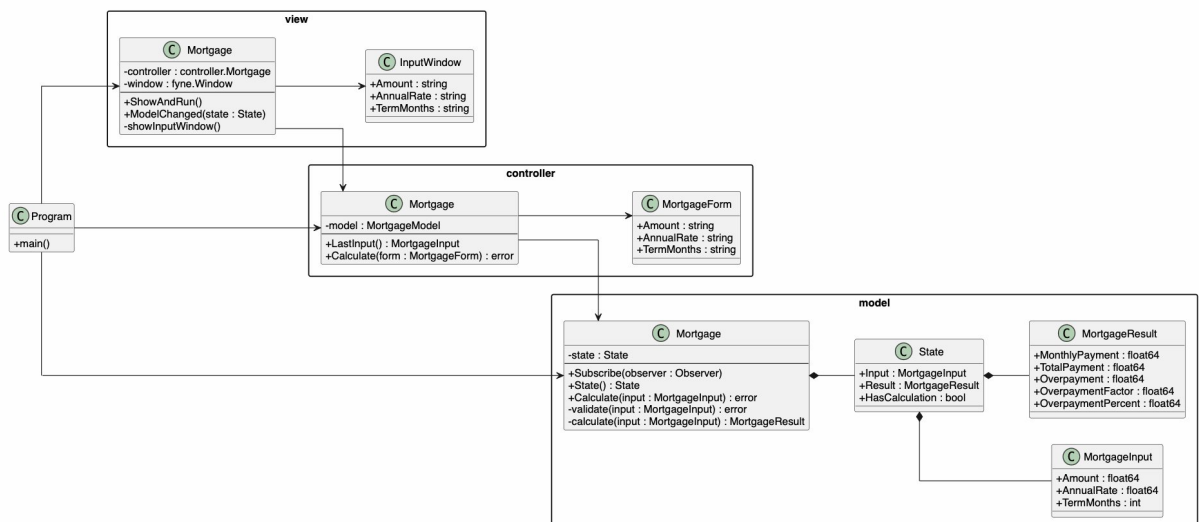


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

3.5 Модель

LoanModel хранит указатели на последние проверенные параметры input и результат result, а также спрез listeners. Метод SetLoanData вызывает Calculate, сохраняет состояние и уведомляет подписчиков через ModelChanged. Функция Calculate не обращается к Fyne и зависит только от переданной структуры LoanInput, поэтому расчёт проверяется обычными тестами Go.

LoanInput и LoanResult передаются как значения. LoanResult содержит ежемесячный платёж, общую сумму выплат, денежную переплату, коэффициент переплаты и число месяцев. При нулевой ставке используется отдельная ветвь, чтобы не делить на ноль.

3.6 Представление

MainView хранит fyne.Window, карту меток результата и обработчик ActionHandler. Главная форма показывает введенные параметры и четыре результата: ежемесячный платёж, общую сумму выплат, переплату и её коэффициент. Метод ModelChanged форматирует деньги и обновляет метки, а ShowError выводит диалог Fyne.

ShowInputDialog создаёт форму Fyne с тремя полями и кнопками «Рассчитать» и «Отмена». Если LoanModel уже содержит ввод, значения подставляются перед показом диалога. Представление передаёт контроллеру исходные строки и не выполняет преобразование или проверку.

3.7 Контроллер

LoanController реализует ActionHandler. Метод EnterDataRequested открывает диалог через MainView, получает строки после подтверждения и вызывает ParseInput. Сумма должна быть больше нуля и не превышать 10^{12} BYN, ставка должна находиться от 0 до 100 %, срок – целое число от 1 до 50 лет. Допускаются точка или запятая как десятичный разделитель и пробелы в сумме.

ParseInput возвращает значение error с конкретным сообщением. Контроллер вызывает view.ShowError и завершает обработчик. Поскольку SetLoanData не вызывается, модель сохраняет предыдущие корректные данные.

3.8 Запоминание данных и обработка ошибок

Запоминание реализуется за счёт хранения структуры MortgageInput внутри состояния модели State. При открытии окна ввода метод LastInput получает последние успешно введенные параметры из модели, после чего поля суммы кредита, процентной ставки и срока заполняются сохранёнными значениями. Обработка ошибок выполняется в два этапа. Сначала контроллер преобразует строки формы в числа с помощью функций parseFloat и parseInt, проверяя наличие и корректность введенных значений. Затем функция validate модели проверяет логические ограничения: сумма кредита должна быть положительной, ставка — находиться в диапазоне от 0 до 100 процентов, а срок — от 1 до 1200 месяцев. При обнаружении ошибки представление выводит соответствующее сообщение с помощью dialog.ShowError.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В результате выполнения лабораторной работы в среде Visual Studio Code разработано приложение на языке Go с интерфейсом Fyne. Приложение построено по паттерну MVC с активной моделью. Проверены все основные сценарии работы приложения: валидация введённых данных, расчёт аннуитетного ежемесячного платежа, общей суммы выплат и переплаты, обработка нулевой процентной ставки, а также восстановление ранее введённых параметров кредита. Экранные формы приведены на рисунках 4.1–4.4, исходный код программы – в приложении А. Рисунок 4.1 – Главная форма после запуска приложения.

The screenshot shows a macOS-style window titled "Кредит на жильё". The interface is dark-themed and contains the following elements:

- Header:** "Кредит на жильё" and a subtitle "Рассчитайте платёжи спокойно и прозрачно".
- Section: Параметры кредита**
 - Subtitle: "Последние введённые данные сохраняются"
 - Inputs: Three input fields labeled "Сумма", "Ставка", and "Срок", each with a placeholder dash "—".
- Section: Ежемесячный платёж**
 - Subtitle: "Аннуитетный платёж"
 - Button: "Введите параметры" (highlighted in blue).
- Summary Section (Three Columns):**
 - Всего банку:** "Сумма выплат за весь срок" with a placeholder "—".
 - Переплата:** "Проценты сверх суммы кредита" with a placeholder "—".
 - Итоговый коэффициент:** "Во сколько раз больше суммы кредита" with a placeholder "—".
- Footer:** A blue button labeled "Ввести данные" in the bottom right corner.

Рисунок 4.1 – Главная форма после запуска

● ● ● Параметры кредита

Новый расчёт

Укажите условия кредита. Дробную ставку можно вводить через запятую или точку.

Сумма кредита, BYN	250000
Годовая ставка, %	14.5
Срок кредита, месяцев	240

Отмена Рассчитать

Рисунок 4.2 – Окно ввода с восстановленными данными

Кредит на жильё

Рассчитайте платёжи спокойно и прозрачно

Параметры кредита

Последние введённые данные сохраняются

Сумма	Ставка	Срок
250 000,00 BYN	14,5%	240 месяцев

Ежемесячный платёж

Аннуитетный платёж

3 199,99 BYN

Всего банку	Переплата	Итоговый коэффициент
Сумма выплат за весь срок	Проценты сверх суммы кредита	Во сколько раз больше суммы кредита
767 998,66 BYN	517 998,66 BYN	3,07 раза

Ввести данные

Рисунок 4.3 – Главная форма с результатом расчёта

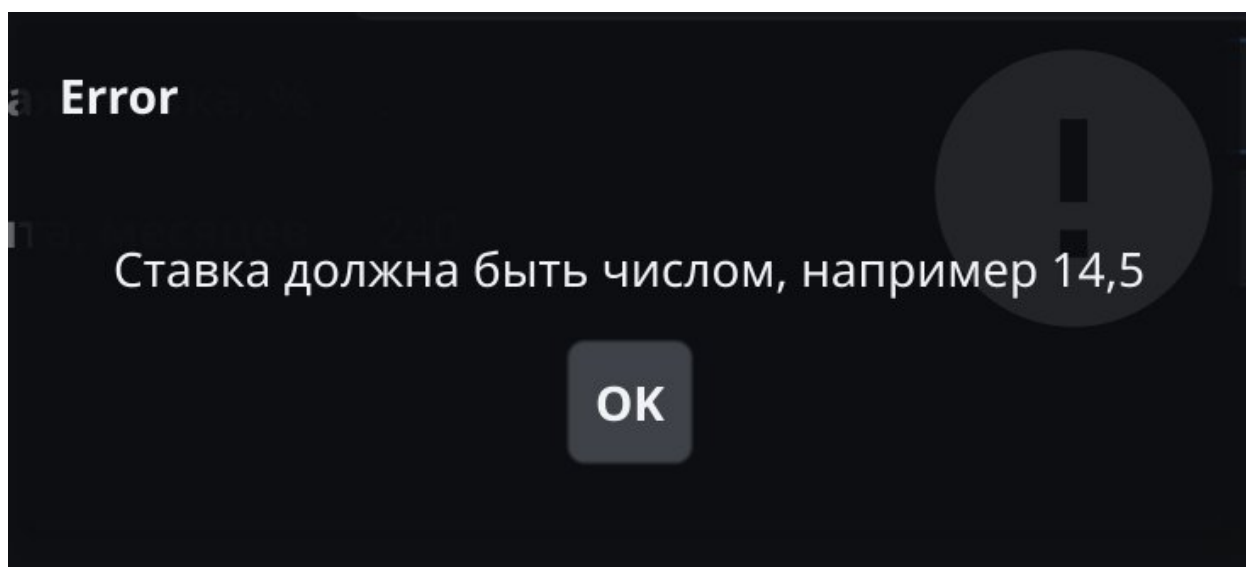


Рисунок 4.4 – Сообщение при некорректной сумме кредита

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки, отладчик, средства запуска, навигация и рефакторинг, интеграция с системой контроля версий, терминал и расширения. В Visual Studio Code эти функции представлены Explorer, Search, Source Control, Run and Debug, Extensions, Problems и Terminal; языковые функции Go предоставляет gopls.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

Основные приёмы включают автодополнение, быстрые исправления, переименование, поиск определений и ссылок, форматирование gofmt, организацию импортов, сворачивание блоков и сравнение версий Git. gopls выполняет фоновую диагностику и отмечает часть ошибок до сборки.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

Команда go build ./... компилирует пакеты модуля, go run . собирает и запускает приложение, go test ./... выполняет тесты. Эти команды запускаются в терминале или задачах Visual Studio Code. Конфигурация Go в launch.json передаёт запуск отладчику Delve.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, разделяющий данные и бизнес-логику, графический интерфейс и обработку действий пользователя. Model хранит состояние и выполняет расчёт. View отображает данные. Controller проверяет ввод и изменяет Model. В работе этими ролями служат LoanModel, MainView с формой Fyne и LoanController.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление: после изменения данных Controller отдельно инициирует обновление View. Активная модель реализует подписку и после изменения сама вызывает наблюдателей. В работе LoanModel хранит спрез Listener и вызывает ModelChanged, а MainView обновляет метки из полученных данных.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные компоненты Visual Studio Code, расширение Go, форматирование gofmt, сборка, тестирование и отладка с Delve. Для представления UML и SysML v2 рассмотрены соответствующие расширения редактора, подготовлены структурная модель и диаграмма типов.

Разработана программа расчёта жилищного кредита на Go с интерфейсом Fyne. Она принимает сумму, годовую ставку и срок, показывает ежемесячный платёж, общую сумму выплат, денежную переплату и коэффициент переплаты. Последние корректные данные восстанавливаются при повторном вводе; пустые значения, неверный формат и выход за допустимые диапазоны сопровождаются сообщениями. LoanModel выполняет расчёт и уведомляет MainView, а LoanController отвечает за проверку ввода и изменение состояния.

ПРИЛОЖЕНИЕ А
обязательное
Листинг исходного кода приложения

```

package main

import (
    "log"

    "github.com/ekuzm/sdlc-lab1/internal/app"
)

func main() {
    if err := app.Run(); err != nil {
        log.Fatalf("run app: %v", err)
    }
}

package app

import (
    "fmt"

    "fyne.io/fyne/v2/app"

    "github.com/ekuzm/sdlc-lab1/internal/controller"
    "github.com/ekuzm/sdlc-lab1/internal/model"
    "github.com/ekuzm/sdlc-lab1/internal/view"
)

func Run() error {
    application := app.NewWithID("by.bsuir.sdlc.mortgage-
calculator")
    mortgageModel := model.NewMortgage()
    mortgageController := controller.NewMortgage(mortgageModel)
    mortgageView := view.NewMortgage(application,
mortgageController)

    mortgageModel.Subscribe(mortgageView)
    mortgageView.ShowAndRun()

    if application.Driver() == nil {
        return fmt.Errorf("application driver is not
initialized")
    }

    return nil
}

package apperror

type Code string

const (
    CodeAmountRequired Code = "AMOUNT_REQUIRED"
    CodeAmountFormat    Code = "AMOUNT_FORMAT"
    CodeAmountPositive  Code = "AMOUNT_POSITIVE"

```

```

        CodeRateRequired    Code = "RATE_REQUIRED"
        CodeRateFormat      Code = "RATE_FORMAT"
        CodeRateRange       Code = "RATE_RANGE"
        CodeTermRequired    Code = "TERM_REQUIRED"
        CodeTermFormat      Code = "TERM_FORMAT"
        CodeTermRange       Code = "TERM_RANGE"
        CodeCalculation     Code = "CALCULATION"
    )

    type Error struct {
        code    Code
        message string
    }

    func New(code Code, message string) *Error {
        return &Error{
            code:    code,
            message: message,
        }
    }

    func (e *Error) Error() string {
        return e.message
    }

    func (e *Error) Code() Code {
        return e.code
    }

    var (
        ErrAmountRequired = New(CodeAmountRequired, "Введите сумму кредита")
        ErrAmountFormat    = New(CodeAmountFormat, "Сумма должна быть числом, например 250000")
        ErrAmountPositive  = New(CodeAmountPositive, "Сумма кредита должна быть больше нуля")
        ErrRateRequired    = New(CodeRateRequired, "Введите процентную ставку")
        ErrRateFormat      = New(CodeRateFormat, "Ставка должна быть числом, например 14,5")
        ErrRateRange       = New(CodeRateRange, "Ставка должна быть от 0 до 100 процентов")
        ErrTermRequired    = New(CodeTermRequired, "Введите срок кредита")
        ErrTermFormat      = New(CodeTermFormat, "Срок должен быть целым числом")
        ErrTermRange       = New(CodeTermRange, "Срок кредита должен составлять от 1 до 1200 месяцев")
        ErrCalculation     = New(CodeCalculation, "Не удалось рассчитать кредит с указанными параметрами")
    )

    package controller

```

```

import (
    "fmt"
    "strconv"
    "strings"

    "github.com/ekuzm/sdlc-lab1/internal/apperror"
    "github.com/ekuzm/sdlc-lab1/internal/model"
)

type MortgageModel interface {
    Calculate(model.MortgageInput) error
    State() model.State
}

type MortgageForm struct {
    Amount      string
    AnnualRate  string
    TermMonths  string
}

type Mortgage struct {
    model MortgageModel
}

func NewMortgage(mortgageModel MortgageModel) *Mortgage {
    return &Mortgage{model: mortgageModel}
}

func (c *Mortgage) LastInput() model.MortgageInput {
    return c.model.State().Input
}

func (c *Mortgage) Calculate(form MortgageForm) error {
    amount, err := strconv.ParseFloat(form.Amount,
apperror.ErrAmountRequired, apperror.ErrAmountFormat)
    if err != nil {
        return fmt.Errorf("parse amount: %w", err)
    }

    rate, err := strconv.ParseFloat(form.AnnualRate,
apperror.ErrRateRequired, apperror.ErrRateFormat)
    if err != nil {
        return fmt.Errorf("parse annual rate: %w", err)
    }

    termMonths, err := strconv.ParseInt(form.TermMonths)
    if err != nil {
        return fmt.Errorf("parse term: %w", err)
    }

    input := model.MortgageInput{
        Amount:      amount,

```

```

        AnnualRate: rate,
        TermMonths: termMonths,
    }
    if err = c.model.Calculate(input); err != nil {
        return fmt.Errorf("calculate mortgage: %w", err)
    }

    return nil
}

func parseFloat(value string, requiredErr, formatErr error)
(float64, error) {
    value = normalizeNumber(value)
    if value == "" {
        return 0, requiredErr
    }

    result, err := strconv.ParseFloat(value, 64)
    if err != nil {
        return 0, formatErr
    }

    return result, nil
}

func parseInt(value string) (int, error) {
    value = strings.TrimSpace(value)
    if value == "" {
        return 0, apperror.ErrTermRequired
    }

    result, err := strconv.Atoi(value)
    if err != nil {
        return 0, apperror.ErrTermFormat
    }

    return result, nil
}

func normalizeNumber(value string) string {
    value = strings.TrimSpace(value)
    value = strings.ReplaceAll(value, " ", "")
    value = strings.ReplaceAll(value, "\u00a0", "")
    value = strings.ReplaceAll(value, ",", ".")

    return value
}

package model

import (
    "math"
    "sync"

```

```

        "github.com/ekuzm/sdlc-lab1/internal/apperror"
    )

    type MortgageInput struct {
        Amount          float64
        AnnualRate      float64
        TermMonths      int
    }

    type MortgageResult struct {
        MonthlyPayment    float64
        TotalPayment      float64
        Overpayment       float64
        OverpaymentFactor float64
        OverpaymentPercent float64
    }

    type State struct {
        Input          MortgageInput
        Result          MortgageResult
        HasCalculation bool
    }

    type Observer interface {
        ModelChanged(State)
    }

    type Mortgage struct {
        mu          sync.RWMutex
        state        State
        observers    []Observer
    }

    func NewMortgage() *Mortgage {
        return &Mortgage{}
    }

    func (m *Mortgage) Subscribe(observer Observer) {
        m.mu.Lock()
        m.observers = append(m.observers, observer)
        state := m.state
        m.mu.Unlock()

        observer.ModelChanged(state)
    }

    func (m *Mortgage) State() State {
        m.mu.RLock()
        defer m.mu.RUnlock()

        return m.state
    }

```

```

func (m *Mortgage) Calculate(input MortgageInput) error {
    if err := validate(input); err != nil {
        return err
    }

    result, err := calculate(input)
    if err != nil {
        return err
    }

    m.mu.Lock()
    m.state = State{
        Input:      input,
        Result:      result,
        HasCalculation: true,
    }
    state := m.state
    observers := append([]Observer(nil), m.observers...)
    m.mu.Unlock()

    for _, observer := range observers {
        observer.ModelChanged(state)
    }

    return nil
}

func validate(input MortgageInput) error {
    if input.Amount <= 0 || math.IsNaN(input.Amount) ||
    math.IsInf(input.Amount, 0) {
        return apperror.ErrAmountPositive
    }

    if input.AnnualRate < 0 || input.AnnualRate > 100 ||
    math.IsNaN(input.AnnualRate) || math.IsInf(input.AnnualRate, 0)
    {
        return apperror.ErrRateRange
    }

    if input.TermMonths < 1 || input.TermMonths > 1200 {
        return apperror.ErrTermRange
    }

    return nil
}

func calculate(input MortgageInput) (MortgageResult, error) {
    months := float64(input.TermMonths)
    monthlyRate := input.AnnualRate / 100 / 12

    monthlyPayment := input.Amount / months
    if monthlyRate > 0 {

```

```

        growth := math.Pow(1+monthlyRate, months)
        monthlyPayment = input.Amount * monthlyRate * growth /
(growth - 1)
    }

    totalPayment := monthlyPayment * months
    overpayment := totalPayment - input.Amount
    result := MortgageResult{
        MonthlyPayment:    monthlyPayment,
        TotalPayment:      totalPayment,
        Overpayment:       overpayment,
        OverpaymentFactor: totalPayment / input.Amount,
        OverpaymentPercent: overpayment / input.Amount * 100,
    }

    if          math.IsNaN(result.MonthlyPayment)          ||
math.IsInf(result.MonthlyPayment, 0) {
        return MortgageResult{}, apperror.ErrCalculation
    }

    return result, nil
}

package view

import (
    "errors"
    "fmt"
    "strconv"
    "strings"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/canvas"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/dialog"
    "fyne.io/fyne/v2/layout"
    "fyne.io/fyne/v2/widget"

    "github.com/ekuzm/sdlc-lab1/internal/apperror"
    "github.com/ekuzm/sdlc-lab1/internal/controller"
    "github.com/ekuzm/sdlc-lab1/internal/model"
)

type Mortgage struct {
    app      fyne.App
    window   fyne.Window
    controller *controller.Mortgage

    amountValue    *widget.Label
    rateValue      *widget.Label
    termValue      *widget.Label
    monthlyValue   *widget.Label
    totalValue     *widget.Label

```

```

        overpaymentValue *widget.Label
        factorValue      *widget.Label
    }

func NewMortgage(application fyne.App, mortgageController
*Mortgage) *Mortgage {
    application.Settings().SetTheme(newModernTheme())

    view := &Mortgage{
        app:      application,
        window:    application.NewWindow("Кредит на жильё"),
        controller: mortgageController,
    }
    view.build()

    return view
}

func (v *Mortgage) ShowAndRun() {
    v.window.ShowAndRun()
}

func (v *Mortgage) ModelChanged(state model.State) {
    if !state.HasCalculation {
        v.renderEmptyState()
        return
    }

    v.amountValue.SetText(formatMoney(state.Input.Amount))
    v.rateValue.SetText(formatPercent(state.Input.AnnualRate))
    v.termValue.SetText(formatTerm(state.Input.TermMonths))
    v.monthlyValue.SetText(formatMoney(state.Result.MonthlyPayment))
    v.totalValue.SetText(formatMoney(state.Result.TotalPayment))
    v.overpaymentValue.SetText(formatMoney(state.Result.Overpayment))
    v.factorValue.SetText(formatFactor(state.Result.OverpaymentFactor))
}

func (v *Mortgage) build() {
    v.window.Resize(fyne.NewSize(960, 650))
    v.window.CenterOnScreen()

    v.amountValue = valueLabel()
    v.rateValue = valueLabel()
    v.termValue = valueLabel()
    v.monthlyValue = heroValueLabel()
    v.totalValue = valueLabel()
    v.overpaymentValue = valueLabel()
    v.factorValue = valueLabel()
}

```

```

    title := widget.NewLabelWithStyle("Кредит на жильё",
fyne.TextAlignLeading, fyne.TextStyle{Bold: true})
    title.TextStyle = fyne.TextStyle{Bold: true}
    subtitle := widget.NewLabel("Рассчитайте платёжи спокойно и
прозрачно")
    subtitle.Importance = widget.LowImportance

    header := container.NewVBox(title, subtitle)

    inputCard := surfaceCard(
        "Параметры кредита",
        "Последние введённые данные сохраняются",
        container.NewGridWithColumns(3,
            metric("Сумма", v.amountValue),
            metric("Ставка", v.rateValue),
            metric("Срок", v.termValue),
        ),
        false,
    )

    monthlyCard := surfaceCard(
        "Ежемесячный платёж",
        "Аннуитетный платёж",
        container.NewCenter(v.monthlyValue),
        true,
    )

    results := container.NewGridWithColumns(3,
        metricCard("Всего банку", "Сумма выплат за весь срок",
v.totalValue),
        metricCard("Переплата", "Проценты сверх суммы кредита",
v.overpaymentValue),
        metricCard("Итоговый коэффициент", "Во сколько раз
больше суммы кредита", v.factorValue),
    )

    calculateButton := widget.NewButton("Ввести данные",
v.showInputWindow)
    calculateButton.Importance = widget.HighImportance

    actions := container.NewBorder(nil, nil, nil,
calculateButton)

    content := container.NewVBox(
        header,
        widget.NewSeparator(),
        inputCard,
        monthlyCard,
        results,
        actions,
    )

```

```

        v.window.SetContent(container.NewPadded(container.NewV
Scroll(content)))
        v.renderEmptyState()
    }

func (v *Mortgage) renderEmptyState() {
    const empty = "-"
    v.amountValue.SetText(empty)
    v.rateValue.SetText(empty)
    v.termValue.SetText(empty)
    v.monthlyValue.SetText("Введите параметры")
    v.totalValue.SetText(empty)
    v.overpaymentValue.SetText(empty)
    v.factorValue.SetText(empty)
}

func (v *Mortgage) showInputWindow() {
    input := v.controller.LastInput()
    inputWindow := v.app.NewWindow("Параметры кредита")
    inputWindow.Resize(fyne.NewSize(560, 440))
    inputWindow.CenterOnScreen()

    amountEntry := widget.NewEntry()
    amountEntry.SetPlaceholder("Например, 250 000")
    rateEntry := widget.NewEntry()
    rateEntry.SetPlaceholder("Например, 14,5")
    termEntry := widget.NewEntry()
    termEntry.SetPlaceholder("Например, 240")

    if input.Amount > 0 {
        amountEntry.SetText(formatInputFloat(input.Amount))
        rateEntry.SetText(formatInputFloat(input.AnnualRate))
        termEntry.SetText(strconv.Itoa(input.TermMonths))
    }

    form := widget.NewForm(
        widget.NewFormItem("Сумма кредита, BYN", amountEntry),
        widget.NewFormItem("Годовая ставка, %", rateEntry),
        widget.NewFormItem("Срок кредита, месяцев", termEntry),
    )

    intro := widget.NewLabel("Укажите условия кредита. Дробную
ставку можно вводить через запятую или точку.")
    intro.Wrapping = fyne.TextWrapWord
    intro.Importance = widget.LowImportance

    cancelButton := widget.NewButton("Отмена",
inputWindow.Close)
    calculateButton := widget.NewButton("Рассчитать", func() {
        err := v.controller.Calculate(controller.MortgageForm{
            Amount: amountEntry.Text,
            AnnualRate: rateEntry.Text,
            TermMonths: termEntry.Text,

```

```

        })
        if err != nil {
            dialog.ShowError(userFacingError(err),
inputWindow)
            return
        }

        inputWindow.Close()
    })
    calculateButton.Importance = widget.HighImportance

    actions := container.NewHBox(layout.NewSpacer(),
cancelButton, calculateButton)
    content := container.NewVBox(
        widget.NewLabelWithStyle("Новый расчёт",
fyne.TextAlignLeading, fyne.TextStyle{Bold: true}),
        intro,
        widget.NewSeparator(),
        form,
        layout.NewSpacer(),
        actions,
    )

    inputWindow.SetContent(container.NewPadded(content))
    inputWindow.Show()
    inputWindow.RequestFocus()
}

func metric(title string, value *widget.Label) fyne.CanvasObject
{
    label := widget.NewLabel(title)
    label.Importance = widget.LowImportance

    return container.NewVBox(
        container.NewCenter(label),
        value,
    )
}

func metricCard(title, subtitle string, value *widget.Label)
fyne.CanvasObject {
    return surfaceCard(title, subtitle,
container.NewCenter(value), false)
}

func surfaceCard(title, subtitle string, content
fyne.CanvasObject, emphasized bool) fyne.CanvasObject {
    backgroundColor := surfaceColor
    borderColor := surfaceBorderColor
    if emphasized {
        backgroundColor = raisedSurfaceColor
        borderColor = accentBorderColor
    }
}

```

```

    background := canvas.NewRectangle(backgroundColor)
    background.CornerRadius = 10
    background.StrokeColor = borderColor
    background.StrokeWidth = 1

    titleLabel      :=      widget.NewLabelWithStyle(title,
fyne.TextAlignLeading, fyne.TextStyle{Bold: true})
    subtitleLabel := widget.NewLabel(subtitle)
    subtitleLabel.Importance = widget.LowImportance

    body      :=      container.NewVBox(titleLabel,      subtitleLabel,
content)
    return      container.NewStack(background,
container.NewPadded(body))
}

func valueLabel() *widget.Label {
    return widget.NewLabelWithStyle("-", fyne.TextAlignCenter,
fyne.TextStyle{Bold: true})
}

func heroValueLabel() *widget.Label {
    label := widget.NewLabelWithStyle("Введите параметры",
fyne.TextAlignCenter, fyne.TextStyle{Bold: true})
    label.Importance = widget.HighImportance

    return label
}

func formatTerm(months int) string {
    return fmt.Sprintf("%d %s", months, plural(months, "месяц",
"месяца", "месяцев"))
}

func plural(number int, one, few, many string) string {
    lastTwo := number % 100
    if lastTwo >= 11 && lastTwo <= 14 {
        return many
    }

    switch number % 10 {
    case 1:
        return one
    case 2, 3, 4:
        return few
    default:
        return many
    }
}

func formatMoney(value float64) string {
    return groupThousands(fmt.Sprintf("%.2f", value)) + " BYN"
}

```

```

}

func formatPercent(value float64) string {
    return strings.ReplaceAll(formatInputFloat(value), ".", ",")
+ "%"
}

func formatFactor(value float64) string {
    return strings.ReplaceAll(fmt.Sprintf("%.2f", value), ".",
",") + " paza"
}

func formatInputFloat(value float64) string {
    return strconv.FormatFloat(value, 'f', -1, 64)
}

func groupThousands(value string) string {
    parts := strings.SplitN(value, ".", 2)
    integer := parts[0]

    for index := len(integer) - 3; index > 0; index -= 3 {
        integer = integer[:index] + " " + integer[index:]
    }

    if len(parts) == 1 {
        return integer
    }

    return integer + "," + parts[1]
}

func userFacingError(err error) error {
    knownErrors := []error{
        apperror.ErrAmountRequired,
        apperror.ErrAmountFormat,
        apperror.ErrAmountPositive,
        apperror.ErrRateRequired,
        apperror.ErrRateFormat,
        apperror.ErrRateRange,
        apperror.ErrTermRequired,
        apperror.ErrTermFormat,
        apperror.ErrTermRange,
        apperror.ErrCalculation,
    }

    for _, knownErr := range knownErrors {
        if errors.Is(err, knownErr) {
            return knownErr
        }
    }

    return apperror.ErrCalculation
}

```

```

package view

import (
    "image/color"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/theme"
)

type modernTheme struct {
    fyne.Theme
}

var (
    appBackgroundColor = color.NRGBA{R: 24, G: 26, B: 29, A: 255}
    surfaceColor       = color.NRGBA{R: 34, G: 37, B: 42, A: 255}
    raisedSurfaceColor = color.NRGBA{R: 40, G: 44, B: 50, A: 255}
    surfaceBorderColor = color.NRGBA{R: 61, G: 66, B: 75, A: 255}
    accentColor        = color.NRGBA{R: 94, G: 156, B: 255, A:
255}
    accentBorderColor  = color.NRGBA{R: 73, G: 125, B: 199, A:
255}
    primaryTextColor   = color.NRGBA{R: 241, G: 243, B: 245, A:
255}
    secondaryTextColor = color.NRGBA{R: 159, G: 166, B: 175, A:
255}
)

func newModernTheme() fyne.Theme {
    return &modernTheme{Theme: theme.DefaultTheme()}
}

func (t *modernTheme) Color(name fyne.ThemeColorName, variant
fyne.ThemeVariant) color.Color {
    switch name {
    case theme.ColorNameBackground:
        return appBackgroundColor
    case theme.ColorNameButton:
        return color.NRGBA{R: 48, G: 52, B: 59, A: 255}
    case theme.ColorNameDisabledButton:
        return color.NRGBA{R: 39, G: 42, B: 47, A: 255}
    case theme.ColorNameDisabled:
        return secondaryTextColor
    case theme.ColorNameInputBackground:
        return color.NRGBA{R: 43, G: 46, B: 52, A: 255}
    case theme.ColorNameInputBorder:
        return surfaceBorderColor
    case theme.ColorNameForeground:
        return primaryTextColor
    case theme.ColorNamePlaceholder:
        return color.NRGBA{R: 133, G: 140, B: 149, A: 255}
    case theme.ColorNamePrimary:

```

```

        return accentColor
    case theme.ColorNameSelection:
        return color.NRGBA{R: 55, G: 91, B: 143, A: 220}
    case theme.ColorNameSeparator:
        return surfaceBorderColor
    case theme.ColorNameHeaderBackground,
theme.ColorNameMenuBackground:
        return surfaceColor
    case theme.ColorNameOverlayBackground:
        return color.NRGBA{R: 12, G: 14, B: 17, A: 210}
    case theme.ColorNameSuccess:
        return color.NRGBA{R: 108, G: 199, B: 154, A: 255}
    case theme.ColorNameWarning:
        return color.NRGBA{R: 224, G: 183, B: 91, A: 255}
    case theme.ColorNameError:
        return color.NRGBA{R: 240, G: 113, B: 120, A: 255}
    case theme.ColorNameForegroundOnPrimary,
        theme.ColorNameForegroundOnSuccess,
        theme.ColorNameForegroundOnWarning,
        theme.ColorNameForegroundOnError:
        return color.NRGBA{R: 248, G: 250, B: 252, A: 255}
    case theme.ColorNameFocus:
        return color.NRGBA{R: 94, G: 156, B: 255, A: 170}
    case theme.ColorNameHover:
        return color.NRGBA{R: 255, G: 255, B: 255, A: 18}
    case theme.ColorNamePressed:
        return color.NRGBA{R: 94, G: 156, B: 255, A: 56}
    case theme.ColorNameHyperlink:
        return accentColor
    case theme.ColorNameScrollBar:
        return color.NRGBA{R: 102, G: 109, B: 119, A: 210}
    case theme.ColorNameScrollBarBackground:
        return color.NRGBA{R: 35, G: 38, B: 43, A: 180}
    case theme.ColorNameShadow:
        return color.NRGBA{R: 7, G: 9, B: 11, A: 110}
    default:
        return t.Theme.Color(name, variant)
}
}

```